

Open Dream-RSI: An Open-Source Library for Recursive Self-Improvement Around a Frozen LLM, with Replay-Gated Learned Policies and a Curated Knowledge Base

Patryk Orwat
Independent Researcher
patrykorwat@gmail.com

September 22, 2026

Abstract

We present **Open Dream-RSI**, an open-source (MIT) Python library that implements the exploration layer of recursive self-improvement (RSI) around a *frozen* large language model: no weight access, no GPU, no training. The library reproduces the discovery loop of Dream-RSI [1] and extends it in three directions that the paper leaves implicit. First, an *always-on supervisor* that self-schedules improvement cycles under an explicit API-call budget guard. Second, *LLM-written exploration policies*: the loop asks the model to write candidate expansion policies as Python source, executes them in an isolated subprocess, scores them by *counterfactual replay rollout* on the recorded discovery tree, and promotes them only on evidence. Third, a *knowledge curator* that distills verifier failures into short, schema-validated, deduplicated lesson records retrieved into every future proposal—so iterations become not only cheaper but measurably smarter. We introduce two deterministic, key-free benchmarks: **bench**, which measures API-call savings (54% at equal solve quality on the built-in suite), and **bench-policy**, a *decoy-trap* suite that isolates exploration quality where solve-rate graphs saturate: score-greedy exploration solves **0/120** tasks, ϵ -greedy 24%, while replay-gated learned policies reach **72%** (100% by cycle 5) at *fewer* total calls than ϵ spends on luck, and the curated knowledge base alone—with zero policy-generation calls—reproduces the same 72%. All headline numbers are produced by a scripted solver and are therefore claims about the *machinery* (gates, sandboxes, promotion, retrieval), not about any particular model; we state this limitation explicitly. The package has zero runtime dependencies, ~ 3.5 k lines of library code, 910 lines of tests (66 passing), a live dashboard, and a single CLI.

Code: <https://github.com/patrykorwat/open-dream-rsi> — MIT license, Python ≥ 3.9 , zero runtime dependencies, installable from source with no build step.

1 Introduction

Recursive self-improvement systems that post-train model weights [2] require GPU clusters, training infrastructure, and weight access. The Dream-RSI line of work [1] shows a lighter recipe: keep the model frozen and instead improve the *process around it*—an offline “dreaming” loop that optimises how the online agent exploits accumulated discovery history. Since the paper’s release (2026-09-14) several independent implementations have appeared; Open Dream-RSI is one of them, not the first (see §6).

This library targets the gap between the paper’s method sketch and a usable tool: a researcher with an OpenAI-compatible endpoint should be able to `pip-install` the loop, point it at their

tasks and model, and get a self-improving search that is (a) budget-safe, (b) safe to run code the model writes, and (c) capable of getting *better*, not merely cheaper. We make three contributions beyond the published method description:

1. **Always-on supervisor with a hard budget guard.** `AutoRSIRuntime` self-schedules cycles, counts every API call (including offline dreaming), and refuses to overspend—the property any unattended deployment needs and no reference implementation ships.
2. **LLM-written exploration policies with replay-gated promotion.** Per task category, the loop asks the frozen model for a policy program (`choose_action(frontier, step)`), statically validates it, executes it in a scrubbed subprocess, scores it against the recorded discovery tree by counterfactual rollout, and promotes it only if it beats the incumbent on evidence (§3). Promotion, not trust, is the safety model.
3. **Knowledge curator: lessons that make iterations smarter.** Recipes and policies capture *what* worked; nothing captured *why* attempts failed, so a trap re-pays its tuition every run. The curator distils verifier failures into short validated lesson records, retrieves them into future proposals, and prunes those that stop helping (§4)—the skills/memory pattern of practical agent frameworks, applied to the loop itself.

Two deterministic, key-free benchmarks back the claims (§5): `bench` measures cost; `bench-policy` measures exploration on a purpose-built decoy-trap suite, because on easy suites every strategy solves everything and solve-rate graphs are blind to policy quality.

2 System architecture

The package is ~3.5k lines of pure-Python with no runtime dependencies (standard library only, including the LLM client) and 910 lines of tests. Modules:

- `core.tree` — discovery tree: hypothesis, verifier score, action history per node.
- `core.agent` — thin LLM-agent abstraction steered by the currently promoted policy.
- `core.simulator` / `core.dreamer` — offline dreaming: replay and optimisation over recorded simulations without touching the online budget.
- `core.policygen` — policy generation, validation, sandboxed execution, counterfactual rollout scoring, promotion gate (§3).
- `core.curator` — lesson distillation, validation, retrieval, pruning (§4).
- `memory` — persistent cross-run store: verified-solution recipes, promoted policies, `lessons.json`.
- `loop` — `AutoRSIRuntime`: scheduling, budget guard, feedback wiring between dreaming and online expansion.
- `llm` — OpenAI-compatible client (POST `{base_url}/chat/completions`) via `urllib`; works against OpenAI, vLLM, Ollama, LM Studio, or any protocol-compatible server through `OPENAI_BASE_URL`.
- `tools` — sandboxed code verification (same isolation boundary as the policy sandbox).
- `dashboard` — dependency-free live web dashboard.

Budget guard. Every call—online proposals, offline dreaming, policy generation, curation—increments a single counter checked before dispatch. A cycle halts at its budget rather than overspending; the supervisor can then sleep, re-plan, and resume from persisted memory. This is what makes the loop deployable unattended: the cost per cycle is a declared quantity, not a hope.

Sandbox boundary. Candidate policy code and model-generated solutions are never executed in-process. Execution happens in `python -I` subprocesses with a scrubbed environment (only `PATH`), a hard timeout (5s per decision), and a temp-dir harness. Invalid code is additionally rejected by AST-level static validation (shape: `def choose_action(frontier, step)`; no imports, no dunder access) before any process spawns. API keys never cross the sandbox wall.

3 LLM-written exploration policies

For each task category the loop may request one LLM call producing a complete policy program. The candidate then passes a pipeline in which every stage can reject it:

1. **Static validation.** AST parse; required entry point present; banned constructs (imports, `eval`, dunder access) reject outright.
2. **Sandboxed execution** against recorded frontiers (above).
3. **Counterfactual rollout scoring.** The candidate’s decisions are replayed on the recorded discovery tree: at each simulated step the candidate picks a frontier node; the reward is the score the child actually achieved in the recorded ladder (back-propagated outcome for never-expanded nodes). One batched subprocess scores an entire rollout ($\sim 15\times$ faster than per-step spawning). The objective is

$$\text{score}(\pi) = \frac{1}{T} \sum_{t=1}^T r_t + \beta_2 \cdot \frac{|\{\text{distinct picks}\}|}{T}, \quad \beta_2 = 0.2, \quad (1)$$

applied *identically* to the candidate and to an in-process greedy baseline. Mean-only scoring mis-ranks explorers below greedy on trap trees; the diversity term is what lets a genuinely structural explorer win the gate.

4. **Promotion gate.** A candidate replaces the incumbent only with evidence: $\geq 80\%$ valid sandbox decisions (`MIN_VALIDITY=0.8`) over a horizon of ≥ 8 recorded steps (`MIN_HORIZON`), and a rollout score beating the greedy baseline under the shared objective.
5. **Call economy.** A category that already solves with its promoted policy is not re-asked; regeneration requires 4 new replay steps of evidence (`POLICY_REGROW_STEPS`), and all generation calls count against the same budget guard.

The promoted program then *steers online expansion*: which frontier node to expand next is decided by validated model-written code rather than a fixed heuristic. This closes the loop the paper describes—the process around the model improves—while every untrusted artefact (code and text alike) passes a structural gate.

4 The knowledge curator

Recipes (verified solutions as warm starts) and policies capture *what* worked. Neither captures *why* attempts fail, so the same trap re-buys its tuition fee every run. The curator adds a third persistent artefact.

Lesson records. After failed verification, the loop invokes the model once (one call, budget-counted) to distill the verifier evidence into lessons carrying a short trigger phrase, a lesson text of ≤ 280 characters, and ≤ 5 evidence snippets.

Lessons are *pure text*—never executed—so their gate is structural plus evidential rather than sandboxed: schema and caps (2–4 trigger words; 20–280 characters; ≤ 8 lessons per category; ≤ 4 lessons / 700 characters injected per prompt; ≤ 5 evidence snippets each), evidence required at admission, and deduplication against the existing store.

Retrieval feedback. At proposal time the top lessons for the category are retrieved (trigger-match, ranked by **wins/uses** counters updated from subsequent verification outcomes); counters drive ranking and eviction, so lessons that demonstrably stop helping are pruned automatically. The knowledge base therefore cannot silently grow stale, and its worst failure mode—injecting an unhelpful paragraph—is bounded by the caps.

What is claimed and what is not. The benchmark below demonstrates that the *wiring* works: a lesson, once admitted, causally changes what the proposal prompt contains and the scripted world responds to it. The quality of lesson *writing* by a live model is an open question this library deliberately does not claim to have settled (§7).

5 Benchmarks

Both benchmarks are deterministic and key-free: a scripted solver makes the model arm a constant, so every delta is attributable to the library’s machinery. They are reproducible from a fresh clone in seconds:

```
python -m open_dream_rsi bench --cycles 10 --markdown
python -m open_dream_rsi bench-policy --cycles 8 --format md
```

5.1 Cost: the dreaming loop against a cold baseline

bench runs the built-in suite (10 cycles $\times$ 5 tasks) twice: a cold baseline (fresh memory each cycle) and the full loop (dreaming + persistent recipes and policies), same scripted model. Measured at the defaults:

arm	solves	API calls	calls / task-cycle
cold baseline (fresh memory each cycle)	50	150	3.00
dream loop (policies + recipes + dreaming)	50	69	1.38

Table 1: **bench**: 54.0% API-call saving at equal solve quality (3,000 dreaming iterations run offline; wall-clock 1.6 s). The curator is disabled in this benchmark to keep the cost measurement pure.

5.2 Exploration: the decoy-trap suite

Solve-rate graphs cannot show that a learned policy helps, because easy suites saturate every arm. **bench-policy** constructs 5 tasks whose fix hides behind a *low-scoring* branch (passes 1/3 tests) past a plausible *decoy* that passes 2/3 forever; family scores are fixed by construction and asserted by tests. Score-greedy expansion locks onto the decoy permanently; escaping requires either structural exploration or memory. Four arms share one scripted solver and budget and differ only in machinery (3 runs $\times$ 8 cycles $\times$ 5 tasks, 24 calls/cycle):

arm	solves	rate	calls	calls/solve	gen. calls
greedy	0/120	0%	480	∞	0
ε -greedy ($\varepsilon=0.3$)	29/120	24%	405	13.97	0
replay-gated policies	86/120	72%	274	3.19	33 policy
curated knowledge base	87/120	72%	283	3.25	40 curator

Table 2: **bench-policy**: the knowledge arm is ε -greedy *with zero policy calls*—every escape above the ε baseline is attributable to remembered lessons. Policies and knowledge reach the same plateau (100% by cycle 5 and 8 respectively); neither is ranked over the other, and 87 vs. 86 is noise.

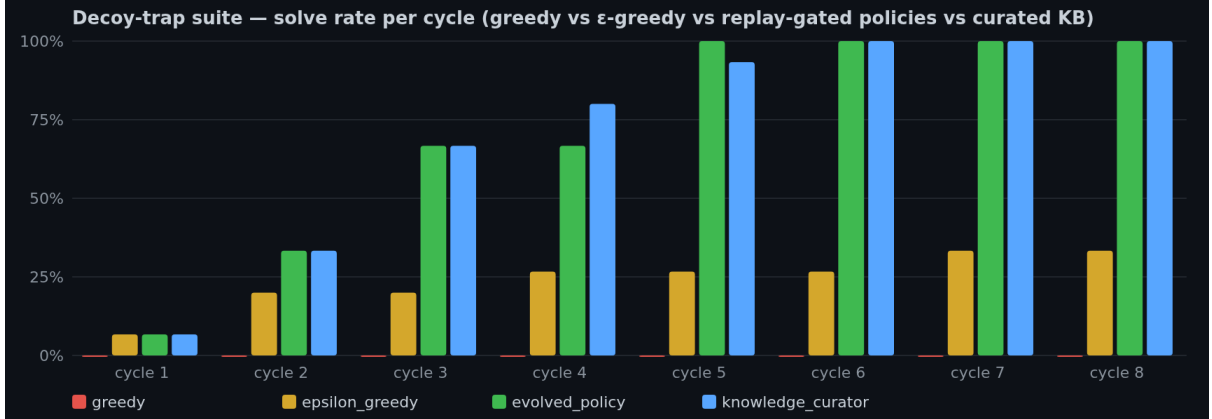


Figure 1: Per-cycle solve rate on the decoy-trap suite (mean over 3 seeds). Greedy never escapes the decoy; ε -greedy escapes by luck; replay-gated policies and the curated knowledge base both saturate at 100%.

Seed robustness. The shipped seeds (7, 11, 23) are not cherry-picked: over five independent seed sets the two knowledge/policy arms land within 78–88 and 78–87 solves of 120 respectively—statistically indistinguishable from each other—while ε -greedy stays at 21–32. The dominance over ε (roughly $3\times$ the solves at $\sim 70\%$ of its calls, 3.2 vs. 14 calls per solve) holds on every set.

5.3 Test suite

66 tests pass (910 LOC), including the benchmark’s behavioural contracts: trap-family scores equal their constructed values; the greedy arm must solve nothing; the gated-policy arm must dominate ε ; curator schema, caps, dedup, and prune logic are pinned by unit tests.

6 Related work and positioning

[1] (Dream-RSI: Recursive Self-Improvement through Evolving Worlds; Zheng et al., Google/DeepMind/UMD, arXiv:2609.14858) is the method this library implements; the official repository is <https://github.com/zhengkid/Dream-RSI>. Independent implementations include a model-agnostic adapter SDK with strict replay and evidence-based promotion gates [3], a local-Qwen CUDA-kernel exploration on DGX Spark [4], and agent/skill variants [5]. Against peers, this library differentiates on: the always-on budget-guarded supervisor; LLM-written policy code (as opposed to a fixed policy menu) with counterfactual-replay promotion; a dedicated exploration benchmark (**bench-policy**); a curated lesson knowledge base with retrieval feedback; persistent cross-run memory; a live dashboard; zero dependencies; MIT license.

A different layer of the same problem is filled by OpenRSI/OpenMLE/Frontis-MA1 [2], which post-train weights (SFT+RL, a 35B meta-evolution agent on MLE-Bench); the two are complementary—their trained improver could be the frozen model behind our client, our dreaming loop could sit on top of their search.

7 Limitations

We state plainly what the numbers do and do not show.

1. **Scripted worlds, not model quality.** Every headline number comes from deterministic benchmarks with a scripted solver; the trap scores are fixed by construction, the scripted solver treats a stored lesson text as a genuine insight when it is visible in the prompt, and the “policy” returned by the scripted model is a canned program. The benchmarks therefore validate the *machinery*—sandboxes, gates, promotion, retrieval, budget—and prove the arms differ *causally* through memory, not that any live model writes good lessons or policies. Live-model evaluation is the required next experiment.
2. **Benchmark self-design risk.** The decoy-trap suite was designed by the same author as the loop it measures. Mitigations: family scores fixed and asserted by construction; arm contracts enforced by tests (greedy must fail, arms must differ causally); determinism lets anyone re-measure with one command; five seed sets reported.
3. **Single-node sandbox.** `python -I` with scrubbed environment and timeout is a real boundary against casual escape but is not a container; deployments with hostile task sources should add OS-level isolation.
4. **Scope.** No weight updates, no multi-agent dynamics, no adversarial red-teaming of promoted policies; the knowledge base is per-category text, not a structured world model.

8 Conclusion

Open Dream-RSI turns the Dream-RSI method into a deployable, budget-safe, dependency-free library and adds the two mechanisms a running loop actually needs to compound—evidence-gated learned policies and a curated, self-pruning knowledge base—each backed by a deterministic benchmark that isolates exactly the capability it claims. At library scale: 54% fewer API calls at equal solve quality; on a suite where greedy exploration solves nothing, replay-gated policies and remembered knowledge both reach 72% cumulative (100% by cycle 5–8) at $\sim 4\times$ better cost-per-solve than random escape. All artefacts pass structural gates before touching the loop; the gate, not trust, decides.

References

- [1] Zheng et al., *Dream-RSI: Recursive Self-Improvement through Evolving Worlds*, arXiv:2609.14858, 2026.
- [2] FrontisAI, *OpenRSI / OpenMLE / Frontis-MA1*, <https://github.com/FrontisAI/OpenRSI>.
- [3] TheAstrayDev, *dream-rsi-sdk*, <https://github.com/TheAstrayDev/dream-rsi-sdk>.
- [4] robinber, *dream-rsi-spark*, <https://github.com/robinber/dream-rsi-spark>.

- [5] lesterppo, *hermes-dream-rsi*, <https://github.com/lesterppo/hermes-dream-rsi>;
Harkit2004, ... *dream-rsi-skill*; mailbobg, *Pi-RSI*.